Executive Summary

We propose a two-phase hackathon centered on “Algorithm ↔ Architecture” co-design. In Phase 1 (Oct 5–13), teams (2–4 members) choose one of five tracks—Quantum Chemistry, Quantum Optimization, Quantum Simulation, Quantum Machine Learning, or Post-Quantum Cryptography—and one problem statement (PS) within that track. For each PS, teams implement a quantum solution (e.g. VQE or QAOA circuit), benchmark it on two fixed online processor models (A and B), analyze how architecture (connectivity, noise, SWAPs) affects performance, then analyze their Phase 1 results and continue with Processor C in the Phase 2 offline challenge. They submit a reproducible GitHub/HF Space repository with code, data, coupling maps, figures and analysis. Submissions undergo automated format checks, an LLM-assisted technical review, and human evaluation. By Oct 17 finalists are announced for Phase 2 (24–26 hr onsite hackathon), where shortlisted teams continue their work in a new, harder offline challenge that introduces Processor C.

Key points: each PS is a research/engineering task with real-world motivation and potential for quantum advantage. For example, computing a molecule’s ground-state energy (quantum chemistry), solving a constrained optimization (finance), or simulating a many-body spin system (physics). Solutions are judged both on problem-specific performance (accuracy, objective value, etc.) and architecture metrics (gate count, depth, SWAPs, fidelity differences). The Universal Challenge Framework (below) fixes Processor A/B specifications and Phase 2 Processor C rules to ensure fairness. This structure is inspired by “Hyperbolic Futures” style challenges that emphasize reproducibility and architectural insight 【28†L99-L107】 【23†L110-L119】.

This document defines: (1) a one-page *Universal Challenge Framework* (processor specs, submission/judging rules), (2) **3 detailed PSs + 1 Open-Innovation prompt** for each of the 5 tracks (total 15 PS + 5 OI), and (3) comparative tables and a participant flow timeline (mermaid diagram). Each PS includes title, scientific problem/motivation (with quantum advantage), baseline task, deliverables, evaluation metrics, data examples, difficulty/effort estimate, and starter hints/references.

Universal Challenge Framework

Phase 1 benchmark authority: Processor A = 5 qubits with coupling edges (0,1), (1,2), (1,3), (3,4). Processor B = 9 qubits with coupling edges (0,1), (1,2), (1,6), (2,3), (2,6), (2,5), (3,4), (4,5), (5,6), (6,7), (6,8), (8,0). These maps are taken from the simulation configuration supplied for the hackathon. The organizer-provided Processor A and Processor B benchmark definitions is the source of truth for the final noise model and simulator settings. Processor C is reserved for Phase 2 offline continuation.

- **Teams:** 2–4 members. Phase 1 individual teams, Phase 2 same teams.
- **Timeline:** Oct 5–8: QFF course & hackathon kickoff. Oct 8–13: online Phase 1 work. Oct 13 (midnight): submission deadline. Oct 14–16: review. Oct 17 (night): finalists announced. **Phase 2**: 24–26 hr onsite hackathon (dates TBA).
- **Submission:** One link to GitHub or Hugging Face Space repository. Repo must contain *README, environment spec, source notebook(s), data/processor files, results/figures,* and a **report** (in notebook or PDF) describing methods and analysis. All code must run end-to-end on provided Qiskit backends.

- **Processor Models for Phase 1 (A/B): Two fixed quantum architectures used for the online qualification benchmark:**

- **Processor A: Fixed 5-qubit online benchmark. Coupling edges: (0,1), (1,2), (1,3), (3,4). The organizer-provided Processor A and Processor B benchmark definitions is authoritative for the exact simulator configuration.**
- **Processor B: Fixed 9-qubit online benchmark with the supplied heavy-hex-inspired coupling graph. Coupling edges: (0,1), (1,2), (1,6), (2,3), (2,6), (2,5), (3,4), (4,5), (5,6), (6,7), (6,8), (8,0). The organizer-provided Processor A and Processor B benchmark definitions is authoritative for the exact simulator configuration.**
- **Processor C: Reserved for the Phase 2 offline continuation. It is not part of the Phase 1 qualification benchmark. Its exact topology and benchmark configuration will be disclosed to shortlisted teams for the onsite challenge.**
- All processors share the same gate set (e.g. X, SX, Rz, CX) and calibration (e.g. two-qubit gate error ~0.5%, single-qubit ~0.1%, see evaluation rubric below).

【35†embed_image】 Figure: Example of a heavy-hex coupling lattice (unit cells). IBM's heavy-hex topology (Processor B) places qubits on hexagon corners and edges to minimize crosstalk【34†L58-L66】.

- **Phase 2 / offline continuation: Processor C will be introduced as the additional architecture for the onsite challenge. The exact Phase 2 task and Processor C configuration will be disclosed to shortlisted teams.**
- **Notebook/Repo Template: Each PS will include a Starter Guide and organizer-provided Processor A and Processor B benchmark definitions: data (molecule integrals, graph data), example code structure, and defined Processor A/B (coupling maps). Teams fill in code blocks. A submission template will list required files:**
- `main.ipynb` (or `.py`) with solution.
- `processors/` folder containing JSON/YAML files for coupling graphs of A, B, C, and custom D.
- `results/` folder for figures/tables.
- `README.md` with instructions to reproduce.
- **Automated Checks:** Submission form will attempt to clone the repo/Space, ensure required files exist, and run a sanity check (e.g. does `main.ipynb` execute without errors and produce specified output variables). Missing items incur penalties.
- **LLM Review:** An LLM will pre-screen each submission for completeness and coherence: e.g. check that the report mentions all required sections (problem statement, approach, results, analysis), verifies format (tables, figures cited), and raises questions (e.g. "Did they explain SWAP overhead?"). The LLM's report (not shown to participants) will highlight omissions for the human judge.
- **Judging Rubric (Phase 1, 100 points):**
- **(40 pts)** *Solution correctness & performance* – Does the code run correctly? Are results (energy, objective, state fidelity) reasonable? Accuracy relative to known references (e.g. classical solution)【28†L99-L107】【23†L110-L119】. Use of algorithms (VQE/QAOA) effectively.
- **(30 pts) Architecture analysis – Quality of benchmarking on A/B. Are architectural metrics (depth, CX count, SWAPs, fidelity) computed? Does the team explain differences between A/B (e.g. "A required 10 swaps vs C's 2 swaps due to its lattice")? Insightfulness of plots and comparisons.**
- **(30 pts) Phase 2 readiness & architecture insight – Quality of the Phase 1 A/B analysis, clarity of identified bottlenecks, and how effectively the team uses those insights in the Phase 2 Processor C continuation.**

- **Bonus/Discretionary:** Clarity of report, quality of visualizations, code organization, and documentation can tip borderline cases.

- **Evaluation Metrics:** Each PS will specify *problem-specific* metrics (e.g. energy error, approximation ratio, fidelity of simulated state) and *common architecture metrics*:

- *Circuit metrics:* total two-qubit gate (CX) count, circuit depth, number of SWAP gates inserted when mapped to each processor.
- *Performance metrics:* ideal result quality (energy error, objective value) and optionally noisy fidelity if noise models are used.
- *Resource metrics:* number of qubits used (some solutions may embed extra ancillas), total single-qubit gates, etc.

- **Finalists (Phase 2):** Top ~10–12 teams per track (depending on interest and capacity) will be invited to the onsite finals. The Phase 2 challenge will build on Phase 1 insights (e.g. a larger instance or new constraint).

Quantum Chemistry Track (VQE/Ab-Initio)

PS-C1: *Molecular Ground State Energy (H_2)*

- **Problem:** Compute the ground-state energy of the H_2 molecule in a minimal basis (STO-3G or similar) using VQE. *Motivation:* Energy calculations of molecules are central to quantum chemistry (protein/drug design, materials) 【28†L93-L101】. VQE provides a near-term approach to these problems (as classical methods scale exponentially). While H_2 is small, this task introduces participants to variational ansätze and hardware limits. In the long term, solving larger molecules could yield quantum advantage in material discovery 【28†L93-L101】.
- **Baseline Task:** Use a standard hardware-efficient or UCCSD ansatz. Compute the ground-state energy at the equilibrium bond length (around 0.74 Å). Verify against the known classical value (chemical accuracy ≈0.0016 Hartree) as a sanity check.
- **Deliverables: A Jupyter notebook implementing VQE for H_2, including (a) Hamiltonian construction (e.g. via Qiskit Nature or precomputed data), (b) ansatz creation, (c) execution on simulators of Processors A/B, (d) results table of energies and resources, (e) coupling-map diagrams (e.g. draw_coupling_map), (f) plotted energy convergence if iterative. Include the fixed architecture definitions for A and B. Ensure repository includes a README, code, and final summary.**
- **Metrics: Accuracy: energy error relative to exact solution. Architecture: number of CX gates and depth on Processors A/B; number of SWAPs needed for mapping; and if noise is enabled, resulting state fidelity or variance. Aim: minimize CX count/depth while reaching chemical accuracy.**

- **Data/Instances:** Prebuilt 4×4 Hamiltonian matrix (or Qiskit operator) for H_2 at R=0.74 Å (and optionally a few near-equilibrium points). No external dataset needed beyond this input. (Provide matrix in files or derive with Qiskit Nature.)
- **Difficulty & Effort:** *Beginner → Medium* (~15–25 hours). This is a classic introductory VQE; participants should be able to follow existing tutorials but must implement it carefully.
- **Hints/Refs: Use Qiskit Nature's FermionicOperator or PauliSumOp for H_2 【28†L93-L101】. See Qiskit's VQE tutorial for H_2 (e.g. Qiskit textbook or docs). As a reference, Harville et al. (2026) survey lists standard VQE ansätze 【28†L99-L107】. Focus on how ansatz depth and mapping affect result on A/B.**

PS-C2: *Diatomic Potential Energy Surface*

- **Problem:** Map out the potential energy curve of H_2 (or LiH) by computing its ground-state energy at multiple bond lengths. *Motivation:* Understanding bond dissociation and reaction pathways is key in chemistry. A quantum computer could, in principle, sample many geometries efficiently. Here, teams will see how ansatz performance changes off-equilibrium (challenging classical approximations). This tests scalability beyond a single point. Quantum advantage: eventually simulating reactive pathways faster than classical electronic structure methods.
- **Baseline Task:** For 5–7 values of bond distance (e.g. 0.5–2.5 Å for H_2), compute ground-state energies via VQE (same ansatz as PS-C1 or improved). Plot energy vs distance. Compare qualitatively to classical curve.
- **Deliverables: Notebook that loops over bond distances, performs VQE at each point, and collects results. Supply final plots of the potential energy curve from Processors A and B. Include tables of (distance, energy_A, energy_B). Visualize resource usage (e.g. CX count vs distance).**
- **Metrics:** *Accuracy:* maximum energy deviation from exact along the curve; ability to capture correct minimum. *Architecture:* how CX count/depth grows with bond length (might grow if effective qubit rotations increase). SWAP overhead differences across processors.
- **Data/Instances:** Provide Hamiltonian data for several bond lengths (Qiskit can generate these from geometry, or give integrals). E.g. H_2 stretch from 0.4 to 2.0 Å.
- **Difficulty & Effort:** *Medium* (~25–35 hours). More coding and computational steps than PS-C1. Teams must optimize the loop and ensure each run is reproducible.
- **Hints/Refs:** Use the same ansatz across distances to isolate hardware impact. Qiskit Nature can generate parameterized molecules. Refer to [14] for ideas on *excited-state VQE* or extensions; but here focus on ground state. Monitoring convergence (optimizer logs) and saving data will help analysis.

PS-C3: *Hardware-Efficient Chemistry*

- **Problem:** Target a slightly larger molecule (e.g. H_2O, HeH^+, or a 3-atom chain like H–H–H) with a **compact ansatz**. Use techniques like qubit tapering, smaller active spaces or adaptive ansatz to reduce resources. *Motivation:* Real molecules have more electrons; we must reduce problem size to fit NISQ devices. Demonstrate creative algorithmic choices. For example, H_2O (10 electrons) is intractable fully; use frozen core or classical pre-screening. Quantum advantage: if we can simulate even modest triatomic molecules with few qubits, that's a step toward chemistry beyond classical limits 【28†L99-L107】.
- **Baseline Task:** Formulate a reduced Hamiltonian for your chosen molecule (e.g. remove core orbitals, freeze spins). Implement either a hardware-efficient ansatz (e.g. layered rotations + entanglers) or an ADAPT/VQE ansatz. Ensure the qubit count ≤8. Run VQE on one bond geometry.
- **Deliverables: Notebook with Hamiltonian setup (document how many qubits and orbitals were used), ansatz design, and results on A/B. Include a table comparing final energies and circuit metrics (depth, CX). Show circuit diagrams for the ansatz. Explain each reduction step (e.g. "we removed orbitals X to fit 8 qubits").**
- **Metrics: Accuracy: ground-state energy error for the reduced model. Resources: number of qubits used, CX count, depth, and SWAPs for Processors A/B. Performance of ansatz (convergence of optimizer).**
- **Data/Instances:** Provide one Hamiltonian for the chosen molecule (e.g. H_2O at equilibrium, reduced to minimal orbitals). Optionally, a list of single-electron integrals.
- **Difficulty & Effort:** *Hard* (~30–45 hours). Requires advanced knowledge of electronic structure (active space selection, qubit mapping). Creativity in ansatz design is essential.
- **Hints/Refs:** Use qubit reduction techniques (e.g. Parity mapping + Z2 symmetries in Qiskit) to cut qubits. ADAPT-VQE (Harville *et al.* surveyed) 【14†L65-L74】 or UCCSD may be too large, so consider hardware-efficient layers. Check examples in literature of small H_2O simulation on 6–8 qubits.

Open Innovation (Chemistry): Participants propose their own quantum chemistry problem. This could be another molecule, an excited-state computation, or computing a molecular property (dipole, reaction rate) on a quantum device. Proposal: By Oct 7, submit a 1–2 page description including the scientific goal, expected algorithm (VQE/QPE/etc.), dataset (e.g. molecule specifications), and why this is challenging/classically intractable. After approval, the participant receives a modified Processor A and Processor B benchmark definitions (same processor models) and continues the Phase 1 tasks as above. (Sample inspiration: optimizing geometry of a transition state, finding binding energies of a ligand-receptor complex, etc.)

Quantum Optimization Track (QAOA/VQE)

PS-O1: *Max-Cut on a Graph*

- **Problem:** Solve the Max-Cut problem on a small weighted graph using QAOA. *Motivation:* Combinatorial optimization (network design, clustering) is a major use-case for quantum algorithms. QAOA is a leading candidate for near-term quantum advantage【23†L110-L119】【24†L54-L63】. This task tests our ability to encode a classical NP-hard problem and see how hardware connectivity affects the algorithm.
- **Baseline Task:** Given a graph of **6 vertices** with random edge weights, formulate the Max-Cut cost Hamiltonian (Z_iZ_j terms). Run QAOA (depth p=1 or 2) to find a cut assignment, using a classical optimizer. Report the best cut and its weight.
- **Deliverables: Notebook that (a) constructs the QUBO for Max-Cut, (b) builds the QAOA circuit, (c) runs the circuit on Processors A/B for several random initial parameters (or uses Qiskit's QAOA module), (d) reports the maximum cut weight found and how close it is to the classical optimum. Include graphs of the random graph (with weights) and coupling diagrams for Processors A/B.**
- **Metrics: Solution quality: approximation ratio = (quantum cut weight) / (optimal cut weight from brute force). Architecture: number of two-qubit gates (CX) required on Processors A/B, depth, and SWAP count due to connectivity differences【23†L124-L134】. Report also the probability of obtaining the optimal bitstring.**
- **Data/Instances:** Provide 2–3 example graphs (in edge-list form) with 6 nodes and integer weights 1–5. Teams can also try one custom graph.
- **Difficulty & Effort:** *Beginner → Medium* (~20–30 hours). QAOA for Max-Cut on 6 nodes is straightforward with Qiskit's tools, but analysis of hardware impact requires careful work.
- **Hints/Refs: Qiskit's Optimization module and QAOA tutorials provide examples of Max-Cut. Farhi et al. (2014) introduced QAOA【23†L110-L119】. Note that QAOA cost terms Z_iZ_j require CNOTs on connected qubits, so limited connectivity (Processor A vs Processor B) leads to SWAP overhead. See Guerreschi & Matsuura (2019) for a Max-Cut QAOA example【23†L110-L119】.**

PS-O2: *Constrained Portfolio Optimization*

- **Problem:** Maximize a return-risk objective in a simple portfolio selection (a QUBO problem). *Motivation:* Portfolio optimization (choosing investments under constraints) is a real-world NP-hard problem. Its formulation as a QUBO has been explored in finance. This PS tests QAOA on a different structure than graph-cut, and with linear constraints (converted via penalties). Potential quantum advantage: solving large finance problems faster than Monte Carlo.

- **Baseline Task:** Given expected returns and covariance (risk) for **4 assets**, formulate a cost function: maximize weighted sum of returns minus λ*(variance), with a budget constraint (exactly k assets chosen). Encode this as a binary quadratic Hamiltonian (using a penalty for the budget constraint). Run a QAOA circuit (p=1) to find an asset selection.
- **Deliverables: Notebook constructing the QUBO matrix (size 4×4) and QAOA circuit. Show classical solver's optimal selection and compare to QAOA solution. Report the chosen portfolio and total objective. Include architecture comparisons for CX count/depth on A/B on Processors A/B.**
- **Metrics: Objective: ratio of quantum solution value to classical optimum. Architecture: as before, CX count, depth, SWAPs on Processors A/B. Also measure how strongly the budget constraint is (frequency of valid solutions).**
- **Data/Instances:** Provide one example: e.g. returns `[5, 3, 4, 2]`, risk matrix (4×4) and budget k=2, penalty λ=2. (Teams can tweak these values for experimentation.)
- **Difficulty & Effort:** *Medium* (~25–35 hours). Requires constructing QUBO from a financial problem, which is more complex than Max-Cut.
- **Hints/Refs:** See Bradley *et al.* (2022) "Portfolio Optimization on a Quantum Computer" for guidance. Qiskit Aqua's Finance module (if available) may have examples. Key idea: add penalty term `A*(Σx_i - k)²` to enforce choosing exactly k assets. Use Qiskit's QAOA or VQE with ZZ couplings to implement the cost 【23†L110-L119】.

PS-O3: ***Knapsack Optimization***

- **Problem:** Solve a 0/1 knapsack problem via a variational quantum approach. *Motivation:* The knapsack problem (select items to maximize value under weight limit) is a canonical NP-hard problem. Encoding it tests ability to handle inequality constraints in QAOA-type circuits. Real-world: resource allocation, cargo loading. Quantum advantage: benchmarking a classic challenge.
- **Baseline Task:** Given **5 items** with values and weights, maximize total value with total weight ≤ capacity. Convert to a cost function with a large penalty for exceeding capacity. Use a shallow QAOA (p=1) ansatz to select items.
- **Deliverables:** Notebook with problem formulation and circuit. Compare QAOA solution to classical brute-force. List which items were chosen. Plot the payoff landscape for different QAOA angles on one processor as illustration.
- **Metrics:** *Solution:* percentage of optimal value reached. *Architecture:* CX count, depth, SWAPs for each topology. Also record how often (probability) a valid (≤capacity) solution is found.
- **Data/Instances:** Provide example: values `[10, 4, 7, 5, 8]`, weights `[5,3,4,2,6]`, capacity 15.
- **Difficulty & Effort:** *Hard* (~30–40 hours). Crafting the cost Hamiltonian with inequality constraint and tuning penalties is nontrivial.

- **Hints/Refs:** Encode the knapsack as a QUBO: `H = -Σ v_i x_i + P·max(0, Σw_i x_i - W)²`. References: Lucas (2014) gives QUBO formulations for knapsack. You may use Qiskit's efficient encoding of `max(0,…)^2` by introducing ancilla or a high penalty P.

Open Innovation (Optimization): Teams define their own optimization problem in any domain (e.g. scheduling, vehicle routing, facility location). Proposal: by Oct 7, submit a 1–2 page problem description: decision variables, objective and constraints, why it's significant, and how to map it to a quantum algorithm (e.g. QAOA, VQE). After review, they proceed as with other PSs (benchmark on A/B, custom architecture, etc.). Example: "Design a quantum algorithm for balanced clustering with at most 3 clusters."

Quantum Simulation Track (Hamiltonian Simulation)

PS-S1: *Ising Model Ground State*

- **Problem:** Find the ground-state energy and magnetization of a 1D Transverse-Field Ising Model (TFIM) chain. *Motivation:* Simulating quantum many-body physics is a hallmark of quantum computing 【28†L93-L101】 【23†L81-L90】. The TFIM exhibits a quantum phase transition; understanding it classically is hard for large size. Here we consider **4 spins** with Hamiltonian $H = -J\sum\sigma_i^z\sigma_{i+1}^z - g\sum\sigma_i^x$. Participants will find the ground state via VQE or QPE. Insights into magnetic ordering have broad impact (quantum magnetism, materials science).
- **Baseline Task:** Using VQE with a suitable ansatz (e.g. hardware-efficient or a checkerboard entangler), estimate the ground-state energy at a given coupling ratio (e.g. J=1, g=1). Also compute expectation $\langle\Sigma\sigma^z\rangle$ (total magnetization).
- **Deliverables: Notebook implementing the TFIM Hamiltonian (4 qubits with periodic or open boundary). Show the exact diagonalization result for comparison. Run the variational algorithm on A/B; report energy, magnetization, and fidelity to exact ground state. Plot magnetization vs external field g (optional).**
- **Metrics:** *Accuracy:* energy error; difference in predicted magnetization vs exact. *Architecture:* circuit CX count and depth for preparing the ansatz state on each device. Number of variational parameters needed.
- **Data/Instances:** Use TFIM with J=1.0, g ∈ {0.5,1.0,1.5}. For each g, prepare the circuit. (Provide the PauliSumOp for H.)
- **Difficulty & Effort:** *Medium* (~25–35 hours). Requires building a spin-chain Hamiltonian (can use Qiskit's `PauliSumOp`) and VQE.
- **Hints/Refs:** The TFIM ground state is fully aligned (↑↑…) at g≪1 and a GHZ-like entangled state at g≫1. One can use a layered ansatz that creates entanglement between neighbors. For reference, see Rind et al. (2022) on VQE for Ising chains.

Trotterized imaginary time evolution (as in Childs *et al.*) is another approach (but VQE is simpler here).

PS-S2: *Real-Time Dynamics*

- **Problem:** Simulate the real-time evolution of a small quantum system. For example, compute the time-dependent expectation of an observable under the TFIM Hamiltonian. *Motivation:* Real-time dynamics (non-equilibrium physics) is intractable classically for many-body systems beyond small sizes【28†L93-L101】. Quantum circuits can directly implement e^{-iHt}. This PS tests Trotterization and error vs cost trade-offs.
- **Baseline Task:** Choose an initial state (e.g. all spins up or a Néel state). Simulate evolution under the same 4-spin TFIM for time t up to $2\hbar/J$, using first-order Trotter steps. Compute $\langle\sigma_1^z\rangle$ or two-point correlators at final time. Compare against exact evolution (via statevector).
- **Deliverables: Notebook performing Trotterized simulation on Processors A/B. Include circuits for one Trotter step, and describe how many steps are used. Plot the observable (e.g. magnetization) vs exact. Show a table of circuit resources (CX, depth) for the simulation on each architecture.**
- **Metrics:** *Accuracy:* absolute error in $\langle\sigma^z\rangle$ (or correlator) vs time. *Architecture:* gates per Trotter step, scaling with number of steps, depth, and SWAPs.
- **Data/Instances:** Provide Hamiltonian and initial state. Suggest 3 different numbers of Trotter steps (e.g. 1, 2, 4) to trade accuracy vs cost.
- **Difficulty & Effort:** *Hard* (~30–40 hours). Circuit depth grows with Trotter steps; careful resource management and analysis is needed.
- **Hints/Refs:** The Trotter circuit for TFIM can be implemented with alternating ZZ interactions and X rotations. See Sadiek *et al.* (2018) for a similar simulation example. Note that each ZZ requires a CNOT pattern; e.g. chain connectivity benefits a linear or heavy-hex layout. Compare first-order vs second-order Trotter (optional).

PS-S3: *Trotter Error vs Noise*

- **Problem:** Investigate the effect of noise on digital quantum simulation accuracy. Starting from PS-S2 circuits, include a realistic noise model (e.g. depolarizing or amplitude-damping on each gate as per hardware parameters). Study how noise-induced error scales with circuit depth and connectivity. *Motivation:* NISQ devices are noisy【50†L345-L354】, so deep Trotter circuits suffer decoherence. Understanding this error is crucial for near-term quantum simulation. This PS engages both simulation and error analysis.
- **Baseline Task: Using the same TFIM setup as PS-S2, pick one evolution scenario (fixed time). Simulate on Processors A/B both with ideal gates and with the organizer-provided Processor A and Processor B benchmark**

definitions noise model (e.g. 0.5% depolarizing on each CX, 0.1% on each single qubit). Compute the difference in final $\langle \sigma^z \rangle$ between ideal and noisy runs.

- **Deliverables: Notebook with noisy simulations on Processors A/B (Qiskit Aer with noise parameters). Plot error vs circuit depth (vary number of Trotter steps). Summarize in a table the fidelity loss or observable error due to noise on Processors A/B.**
- **Metrics:** *Noise impact:* difference in observable due to noise, or worst-case fidelity. *Architecture:* how noise scales with connectivity (e.g. a 2D connectivity might require fewer SWAPs and hence lower noise accumulation). Gate counts as before.
- **Data/Instances:** Use same Hamiltonian and Trotter steps. Define a simple noise model (e.g. depolarizing probability p=0.005 per CX).
- **Difficulty & Effort:** *Hard* (~30–40 hours). Requires implementing noise simulations and interpreting results.
- **Hints/Refs:** Qiskit Aer allows custom noise models. See Appleby *et al.* (2024) on noise in Trotter simulations. Graph both ideal and noisy observables for comparison.

Open Innovation (Simulation): Teams propose their own quantum simulation scenario. Examples: simulating a small molecule's real-time wavefunction (Schrödinger eqn for hydrogen atom), simulating a chemical reaction (e.g. H_2O splitting), or benchmarking a quantum subroutine (e.g. QSVT on a simple matrix). **Proposal:** by Oct 7, submit a short description of the physical system and observable of interest, why it's hard classically, and how to encode it (Hamiltonian, initial state, etc.). Upon approval, participants proceed with tasks 1–7.

Quantum Machine Learning Track (Variational QML)

This track uses the same Universal Challenge Framework as the other tracks: variational circuits are benchmarked as-is on Processors A/B, and teams design a Phase 2 Processor C. The only difference is the learning objective (a trained classifier) sitting on top of the circuit, so both problem-specific ML metrics and architecture metrics apply to every PS below.

PS-Q1: *Biomedical Signal Classifier*

- **Problem:** Classify short ECG or EEG signal segments as normal vs. abnormal using a hybrid quantum-classical variational classifier (VQC). Motivation: automated screening tools support earlier detection of cardiac and neurological conditions, and this PS introduces participants to encoding real, noisy sensor data into a small circuit. Quantum advantage: an open question here is whether the quantum feature space separates classes that a comparable classical model cannot.
- **Baseline Task: Extract 4-8 numeric features per signal segment (e.g. heart rate, beat-to-beat variability, peak amplitude), encode them into a 4-8 qubit VQC via angle encoding, train the circuit parameters with a classical optimizer, and run**

the trained circuit on Processors A/B. Compare against logistic regression on the same features.

- **Deliverables: Notebook covering feature extraction, VQC construction, training loop, and evaluation. Results table (accuracy, F1, recall) per processor, circuit-resource table (CX count, depth, SWAPs) per processor, coupling-map diagrams, and a Phase 2 Processor C design with justification and re-run results.**
- **Metrics:** Accuracy, F1 score and recall (recall is the primary metric, since missing an abnormal case is costlier than a false alarm). Architecture: CX count, depth and SWAPs per processor.
- **Data/Instances:** MIT-BIH Arrhythmia Database (PhysioNet) or a similar public ECG/EEG set, reduced to 200-500 labelled samples with a fixed train/test split provided by organisers.
- **Difficulty & Effort:** Beginner to Medium (~15-20 hours). Standard VQC pipeline; most of the effort is feature engineering and honest baseline comparison.
- **Hints/Refs:** Use Qiskit's EstimatorQNN or PennyLane's qml.qnn.TorchLayer rather than writing a training loop from scratch. Normalize features to [0, pi] before angle encoding. Use class-weighted loss if abnormal cases are underrepresented.

PS-Q2: *Quantum Kernel Classifier*

- **Problem:** Build a quantum kernel from circuit state overlaps and use it inside a classical support vector machine. Motivation: quantum kernels are one of the most rigorously studied candidates for a provable quantum advantage in machine learning, but most real datasets show no advantage once the classical kernel is properly tuned. This PS asks teams to test that claim honestly.
- **Baseline Task: Implement a quantum feature map (e.g. ZZFeatureMap or an equivalent) to compute a kernel matrix, feed it into an SVM, and compare against RBF and polynomial kernels tuned via cross-validation. Run the kernel-evaluation circuits on Processors A/B to capture their resource cost.**
- **Deliverables: Notebook with kernel construction, SVM results, and a statistical comparison across multiple random seeds with confidence intervals. Circuit-resource table (CX, depth, SWAPs, total circuit executions) per processor, and a Phase 2 Processor C proposal aimed at reducing that cost.**
- **Metrics:** Classification accuracy/AUC with confidence intervals across seeds. Architecture: CX count, depth and SWAPs per processor for the kernel-evaluation circuit, and total circuit executions needed.
- **Data/Instances:** 2-3 datasets: one standard benchmark (e.g. a digits subset) and one synthetic dataset constructed so its structure aligns with the chosen quantum feature map, at 4-8 qubits.
- **Difficulty & Effort:** Medium to Hard (~25-35 hours). The circuit itself is simple; the rigor is in the baseline tuning and the statistics.

- **Hints/Refs:** See Havlicek et al. (2019) on quantum kernel methods. Watch for kernel concentration, where similarity values collapse toward the same number as qubit count grows, making the kernel uninformative. Qiskit's QuantumKernel class and PennyLane's kernel utilities cover the mechanics.

PS-Q3: *Noise-Resilient Variational Classifier*

- **Problem:** A classifier trained on an ideal simulator can lose significant accuracy once evaluated under real device noise. Train a classifier and recover its accuracy under a calibrated noise model using error mitigation. Motivation: this is the practical gap between NISQ-era QML research and any real deployment.
- **Baseline Task:** Train a VQC on Processor A's ideal simulator, then evaluate the same trained circuit on Processor B and C using noise models derived from their calibration data. Apply at least two mitigation techniques (e.g. zero-noise extrapolation and readout error correction) and report recovered accuracy.
- **Deliverables: Notebook with ideal, noisy and mitigated results; mitigation implementation; a table of the extra circuit executions each technique costs; and a Phase 2 Processor C proposal aimed at improving noise resilience (e.g. a shallower ansatz with fewer two-qubit gates), with results on it.**
- **Metrics:** Accuracy in the ideal, noisy and mitigated settings; total circuit executions/shots used; variance across repeated runs (a single noisy run is not reliable evidence).
- **Data/Instances:** Reuse the dataset and trained model from PS-Q1 or PS-Q2. Use Qiskit fake-backend noise models mapped onto the Processor B and C specifications.
- **Difficulty & Effort:** Hard (~30-40 hours). Requires understanding both noise modelling and mitigation cost trade-offs, not just running a mitigation library call.
- **Hints/Refs:** The Mitiq library has ready-made zero-noise extrapolation. Readout correction needs a calibration matrix measured once and reused. Watch for barren-plateau effects if scaling qubit count.

Open Innovation (Quantum Machine Learning)

Teams propose their own QML application (e.g. fraud detection on transaction data, a quantum generative model for synthetic tabular data, or a materials-property predictor). Proposal: by Oct 7, submit a 1-2 page description covering the dataset, the quantum model design, the planned classical baseline, and why the task is classically non-trivial at the chosen scale. After approval, the team receives a Processor A and Processor B benchmark definitions (Processor A/B definitions) and proceeds through the standard Phase 1 tasks, including designing Phase 2 Processor C.

Post-Quantum Cryptography Track (Quantum-Safe Systems)

Framework note: PQC problem statements do not run quantum circuits. The algorithms are classical software, designed to remain secure against a future quantum adversary, and they execute entirely on classical hardware. To keep the same three-fixed-points-plus-custom-design structure as the other tracks, this track replaces quantum Processors A/B with three fixed Target Platforms, and Processor D with a custom Platform D.

Platform A: resource-rich baseline. A standard x86-64 laptop or cloud VM, single core, no hardware acceleration.

Platform B: constrained server. Same architecture as A, capped to 1 CPU core and limited RAM, representative of a typical deployed backend service rather than a development machine.

Platform C: embedded/IoT target. An ARM Cortex-M-class microcontroller emulator (e.g. QEMU) or a Raspberry Pi Zero-class device, representative of constrained endpoints such as IoT sensors or smart cards.

Custom Platform D (innovation): after benchmarking on A/B, teams propose a Platform D, i.e. a specific memory/CPU budget or parameter-set choice (e.g. ML-KEM-512 vs. ML-KEM-768) chosen to expose a genuine trade-off in their algorithm, and justify the choice with a re-run of their benchmark on it.

Judging Rubric (Phase 1, 100 points, reinterpreted for this track): (40 pts) Solution correctness & security - does the implementation use standardized algorithms (ML-KEM/ML-DSA/SLH-DSA) correctly, and does it hold up against the stated threat model? (30 pts) Platform analysis - quality of benchmarking across Platform A/B and insight into why results differ. (30 pts) Innovation & custom design - creativity and justification of Platform D, and whether it demonstrates a genuine trade-off.

PS-P1: *Quantum-Safe Secure Channel*

- **Problem:** Build a messaging or file-transfer application secured with ML-KEM (FIPS 203) for key exchange and ML-DSA (FIPS 204) for authentication. Motivation: RSA and elliptic-curve key exchange would be broken by Shor's algorithm on a cryptographically-relevant quantum computer, and this migration to standardized post-quantum primitives is already underway industry-wide.
- **Baseline Task: Implement a handshake using a vetted library (e.g. liboqs): ML-KEM encapsulation for a shared secret, an ML-DSA signature for authentication, HKDF to derive a symmetric key, and AES-256-GCM to encrypt the payload. Benchmark the handshake on Platform A/B.**

- **Deliverables:** Working demo, a flow diagram, a benchmark table of handshake time and memory on each platform, a discussion of the man-in-the-middle risk and how the signature mitigates it, and a custom Platform D benchmark.
- **Metrics:** Correctness (successful, interoperable handshake between two peers); handshake time, memory and code size per platform; total bytes exchanged.
- **Data/Instances:** No external dataset. Organisers provide sample payloads of varying size (1 KB, 100 KB, 10 MB) to test with.
- **Difficulty & Effort:** Beginner to Medium (~15-20 hours). Most of the effort is correct library usage and clear benchmarking, not novel cryptography - teams must not write their own primitives.
- **Hints/Refs:** liboqs-python provides ready KeyEncapsulation and Signature classes. See NIST FIPS 203/204/205 for the standardized algorithms and parameter sets.

PS-P2: ***Crypto Inventory & Migration Planner***

- **Problem:** Build a scanner that finds classical (RSA/elliptic-curve) cryptography in a codebase, rates its quantum risk, and proposes a post-quantum replacement. Motivation: no organisation can migrate cryptography it has not found first, so discovery tooling is the practical first step of any real migration.
- **Baseline Task:** Statically scan a provided sample repository, detect cryptographic API usage patterns, and output a risk-rated report: RSA/elliptic-curve usage is high risk (Shor's algorithm), AES-128 is moderate risk (Grover's algorithm), AES-256 and SHA-256 are lower risk. Map each finding to a suggested replacement (ML-KEM, ML-DSA).
- **Deliverables: The scanning tool, a report on the provided sample project, a measured false-positive/false-negative rate against the organisers' ground truth, and a benchmark of scan time on Platform A/B (e.g. a full repo scan vs. the same scan on a constrained CI runner).**
- **Metrics:** Detection precision and recall against the organisers' ground truth; scan time and memory per platform.
- **Data/Instances:** An organiser-provided multi-language sample repository (Python/Java/C) with planted cryptographic usages; the ground-truth list is withheld from teams until judging.
- **Difficulty & Effort:** Medium (~20-25 hours). Straightforward pattern matching gets partial credit; AST-based detection with a low false-positive rate is where teams differentiate themselves.
- **Hints/Refs:** Regex-based detection is a reasonable start; AST-based tools such as Semgrep rules catch more real usage. See NIST SP 1800-38 for migration-planning guidance.

PS-P3: ***PQC Performance Benchmark Across Constrained Platforms***

- **Problem:** Measure how ML-KEM, ML-DSA and SLH-DSA compare with RSA and elliptic-curve cryptography across a resource-rich server, a constrained server, and an embedded/IoT target. Motivation: post-quantum algorithms have very different key, ciphertext and signature sizes, and those differences can break assumptions in constrained real-world deployments.
- **Baseline Task:** Benchmark key/ciphertext/signature sizes and operation times/memory for at least one parameter set per algorithm (ML-KEM-768, ML-DSA-65, SLH-DSA-128s) against RSA-2048 and P-256, on all three platforms, with at least 100 repetitions per measurement.
- **Deliverables:** Reproducible benchmark scripts for each platform, comparison tables and at least three charts, a written trade-off conclusion, and a proposed custom Platform D chosen to stress-test whichever algorithm showed the weakest results, re-run there.
- **Metrics:** Size in bytes, time in milliseconds, and memory in kilobytes per operation, per platform; the degradation ratio between Platform C and Platform A for each algorithm.
- **Data/Instances:** No external dataset. Organisers fix the parameter sets to keep comparisons fair across teams.
- **Difficulty & Effort:** Medium to Hard (~25-35 hours). Getting a fair, reproducible measurement on three different platforms (including emulated hardware) is the real challenge, not the cryptography itself.
- **Hints/Refs:** liboqs ships a benchmarking harness. QEMU can emulate a Cortex-M target for Platform C. Use a stable timing library (e.g. pyperf) rather than naive wall-clock timing.

Open Innovation (Post-Quantum Cryptography)

Teams propose their own PQC-related challenge, for example a hybrid TLS 1.3 handshake tested under simulated network constraints, a timing-side-channel audit and hardening of a PQC implementation, or a harvest-now-decrypt-later migration-risk model for a synthetic enterprise. Proposal: by Oct 7, submit a 1-2 page description of the system, the threat model, the algorithm(s) involved, and why current, classical-only practice is insufficient. After approval, the team proceeds through the standard Platform A/B/D benchmarking described above.

Comparative PS Overview

Track	PS Title	Impact	Difficulty	Skills
Chemistry	C1: H_2 Ground-State Energy	Introduces quantum chemistry VQE; fundamental test of chemical accuracy 【28†L99-L107】.	Easy–Med	VQE ansatz design, mapping, chemistry basics

Track	PS Title	Impact	Difficulty	Skills
	C2: H_2 Potential Curve	Realistic molecular data (bond stretching); tests solver robustness.	Med	Parameter scanning, VQE loops
	C3: Hardware-Efficient H_2O	Bridges toward realistic molecules; important for scaling to real chemistry.	Hard	Active space reduction, advanced ansatz
	Open: Custom chem. problem	Encourages creativity (new molecules, excited states, etc.).	Hard	Same as above
Optimization	O1: Max-Cut	Standard benchmark; well-known QAOA test for graphs【23†L110-L119】.	Easy–Med	QAOA/QUBO, graph handling
	O2: Portfolio Selection	Practical finance problem with constraints.	Med	QUBO formulation, constraint encoding
	O3: Knapsack	Classic NP-hard; introduces inequality constraints in circuits.	Hard	Advanced QUBO encoding, penalty tuning
	Open: Custom optimization	Applies to logistics, ML, etc. (design own QUBO).	Hard	Problem modeling, QAOA/VQE
Simulation	S1: Ising Chain Ground State	Core many-body problem; demonstrates phase-transition physics.	Med	Hamiltonian building, VQE
	S2: Spin Dynamics (Real-Time)	Non-equilibrium physics; relevant to materials and quantum hardware validation.	Hard	Trotterization, time-evolution

Track	PS Title	Impact	Difficulty	Skills
	S3: Noise in Trotter Simulation	Addresses real NISQ challenge (noise trade-off).	Hard	Noise modeling, error analysis
	Open: Custom simulation	E.g. molecular dynamics, field simulation.	Hard	Physical modeling, simulation
QML	Q1: Biomedical Signal Classifier	Near-term diagnostic-support use case; tests hybrid quantum-classical training.	Easy-Med	VQC design, feature encoding, classical ML basics
	Q2: Quantum Kernel Classifier	Tests the open question of quantum kernel advantage against a tuned classical baseline.	Med-Hard	Kernel methods, SVMs, statistical rigor
	Q3: Noise-Resilient Classifier	Addresses a real NISQ challenge: recovering accuracy from calibrated device noise.	Hard	Error mitigation, noise modeling
	Open: Custom QML problem	Encourages creativity (new datasets, generative models, etc.).	Hard	Same as above
PQC	P1: Quantum-Safe Secure Channel	Practical migration demo; ML-KEM + ML-DSA replacing an RSA/ECC handshake.	Easy-Med	Applied cryptography, secure protocol design
	P2: Crypto Inventory & Migration Planner	Real-world first step of any PQC migration; static-analysis tooling.	Med	Static analysis, risk assessment
	P3: PQC Performance Benchmark	Quantifies real deployment trade-offs across server, edge and IoT platforms.	Med-Hard	Benchmarking, systems measurement

Track	PS Title	Impact	Difficulty	Skills
	Open: Custom PQC problem	E.g. hybrid TLS, side-channel audit, migration risk planning.	Hard	Protocol/security engineering

Participant Flow and Timeline

```
flowchart TD
    A[Start: QFF Fall Fest Course] --> B[Hackathon Kickoff (Oct 5)]
    B --> C{Choose Track}
    C --> C1[Quantum Chemistry]
    C --> C2[Quantum Optimization]
    C --> C3[Quantum Simulation]
    C --> C4[Quantum Machine Learning]
    C --> C5[Post-Quantum Cryptography]
    C1 --> D1[Select PS (C1-C3 or Open)]
    C2 --> D2[Select PS (O1-O3 or Open)]
    C3 --> D3[Select PS (S1-S3 or Open)]
    C4 --> D4[Select PS (Q1-Q3 or Open)]
    C5 --> D5[Select PS (P1-P3 or Open)]
    D1 --> E1[Receive Starter Guide + organizer-provided Processor A and Proc
essor B benchmark definitions]
    D2 --> E2[Review the provided Processor A/B benchmarks]
    D3 --> E3[Review the provided Processor A/B benchmarks]
    D4 --> E4[Review the provided Processor A/B benchmarks]
    D5 --> E5[Review the provided Processor A/B benchmarks]
    E1 --> F1[Solve PS (Core problem)]
    E2 --> F2[Solve PS]
    E3 --> F3[Solve PS]
    E4 --> F4[Solve PS]
    E5 --> F5[Solve PS]
    F1 --> G1[Run on Processor A]
    F2 --> G2[Run on Processor A]
    F3 --> G3[Run on Processor A]
    F4 --> G4[Run on Processor A]
    F5 --> G5[Run on Platform A]
    G1 --> H1[Run on Processor B]
    G2 --> H2[Run on Processor B]
    G3 --> H3[Run on Processor B]
    G4 --> H4[Run on Processor B]
    G5 --> H5[Run on Platform B]
    H1 --> I1[Proceed to Phase 2 offline continuation with Processor C]
    H2 --> I2[Proceed to Phase 2 offline continuation with Processor C]
    H3 --> I3[Proceed to Phase 2 offline continuation with Processor C]
    H4 --> I4[Proceed to Phase 2 offline continuation with Processor C]
    H5 --> I5[Run on Platform C]
    I1 --> J1[Analyze A/B Results & Architecture]
```

```
I2 --> J2[Analyze A/B Results & Architecture]
I3 --> J3[Analyze A/B Results & Architecture]
I4 --> J4[Analyze A/B Results & Architecture]
I5 --> J5[Analyze Results & Platform Trade-offs]
J1 --> K1[Complete Phase 1 architecture analysis]
J2 --> K2[Complete Phase 1 architecture analysis]
J3 --> K3[Complete Phase 1 architecture analysis]
J4 --> K4[Complete Phase 1 architecture analysis]
J5 --> K5[Design Custom Platform D]
K1 --> L1[Prepare Phase 1 Submission]
K2 --> L2[Prepare Phase 1 Submission]
K3 --> L3[Prepare Phase 1 Submission]
K4 --> L4[Prepare Phase 1 Submission]
K5 --> L5[Test Platform D]
L1 --> M1[Prepare Submission (Oct 13)]
L2 --> M2[Prepare Submission]
L3 --> M3[Prepare Submission]
L4 --> M4[Prepare Submission]
L5 --> M5[Prepare Submission]
M1 --> N1[Automated & LLM Review (Oct 14-16)]
M2 --> N2[Automated & LLM Review]
M3 --> N3[Automated & LLM Review]
M4 --> N4[Automated & LLM Review]
M5 --> N5[Automated & LLM Review]
N1 --> O1[Human Judge Scoring]
N2 --> O2[Human Judge Scoring]
N3 --> O3[Human Judge Scoring]
N4 --> O4[Human Judge Scoring]
N5 --> O5[Human Judge Scoring]
O1 --> P[Shortlist Finalists (Oct 17)]
O2 --> P
O3 --> P
O4 --> P
O5 --> P
P --> Q[Phase 2: 24–26hr Onsite Hackathon]
Q --> R[Final Demo & Judging]
R --> S[🏆 Winners Announced]
```

- **Oct 5–8:** Participants study the course material while choosing their track and PS, and set up dev environment.
- **Oct 9–12: Active development: implement the core solution, benchmark A/B, analyze results, and prepare the Phase 1 submission.**
- **Oct 13:** Submit by 11:59 PM.
- **Oct 14–16:** Repo cloning, automated tests, LLM and human review.
- **Oct 17:** Results announced; finalists prepare travel for Phase 2.

References

- Harville *et al.*, *Recent Developments in VQE* (2026) – Survey of VQE methods and NISQ chemistry【28†L99-L107】.
- Coqa (2024) – QAOA compiler optimizations; context on NISQ variational algorithms【23†L110-L119】.
- He *et al.*, *QAOA parameter heuristics* (2024) – Background on QAOA and resource estimates【24†L54-L63】.
- Paul Nation *et al.*, *IBM heavy-hex lattice* (2021) – Explanation of IBM's heavy-hex topology【34†L58-L66】.